関数仕様書

ProtocolMessage モジュール

May 31, 2025

Contents

1 概要 **2**

2 備考 **2**

3 名前空間 `Protocol::CRC` **2**
3.1 uint16_t calculate(const uint8_t* data, size_t length) 2

4 構造体 `Protocol::ProtocolMessage` **2**
4.1 コンストラクタ `ProtocolMessage(uint8_t f, uint32_t ts, uint8_t d)` 2
4.2 デフォルトコンストラクタ `ProtocolMessage()` 3
4.3 void fromBytes(const uint8_t* buf) 3
4.4 void toBytes(uint8_t* buf) const 3

5 名前空間 `Protocol::Factory` **3**
5.1 void createButtonPressMessage(uint8_t* buffer, uint32_t time) 3
5.2 void createLEDStateRequest(uint8_t* buffer) 3
5.3 void createLEDControlResponse(uint8_t* buffer, uint32_t time, bool turnOn) 4
5.4 void createTimeSyncRequest(uint8_t* buffer, uint32_t t1) 4
5.5 void createTimeSyncResponse(uint8_t* buffer, uint32_t t3, uint8_t diff) . . 4

6 名前空間 `Protocol::Parser` **4**
6.1 enum class ParseResult 4
6.2 ParseResult tryParseMessageFromBuffer(const uint8_t* buffer, size_t len, ProtocolMessage& m
6.3 MessageType getMessageType(const ProtocolMessage& msg) 5

1 概要

Protocol::ProtocolMessage の関数仕様をまとめた．メッセージの生成，解析，CRC 計算などの機能を提供する．

2 備考

いろいろチャッピー使って書かせたんだけど，結局使うのはビット配列を作成する関数と，送られてきたデータが欠けていないか確認する関数と，送られてきたデータが何を表しているかを返す関数の三つです．

1. 名前空間 Protocol::Factory に送るビット配列作成用の関数が書かれてます．
2. 名前空間 Protocol::Parser に送られてきたデータのエラー検知と，内容判断の関数が書かれています．

おそらくこれらの関数を使ってもらうと思います．

3 名前空間 Protocol::CRC

3.1 uint16_t calculate(const uint8_t* data, size_t length)

- **概要**：CRC-16-CCITT アルゴリズムに基づき，指定されたバイト列の CRC 値を計算する．
- **引数**：
  - const uint8_t* data：CRC 計算対象のバイト列の先頭ポインタ
  - size_t length：バイト列の長さ
- **戻り値**：計算された 16 ビット CRC 値
- **備考**：初期値 0xFFFF，多項式 0x1021 を使用．

4 構造体 Protocol::ProtocolMessage

4.1 コンストラクタ ProtocolMessage(uint8_t f, uint32_t ts, uint8_t d)

- **概要**：指定されたフラグ，タイムスタンプ，データでメッセージを初期化する．
- **引数**：
  - uint8_t f：メッセージの種類を示すフラグ
  - uint32_t ts：タイムスタンプ
  - uint8_t d：データ領域（ON/OFF や時刻差など）
- **戻り値**：なし

4.2 デフォルトコンストラクタ `ProtocolMessage()`

- **概要**：メンバーをすべて 0 で初期化する.
- **引数**：なし
- **戻り値**：なし

4.3 void fromBytes(const uint8_t* buf)

- **概要**：バイト列（8 バイト）からメッセージデータを展開する.
- **引数**：
  - `const uint8_t* buf`：展開元の 8 バイトのバッファ
- **戻り値**：なし
- **備考**：バイト列は flag(1byte), timestamp(4byte), data(1byte), crc(2byte) の順で格納されている.

4.4 void toBytes(uint8_t* buf) const

- **概要**：メッセージをバイト列（8 バイト）に変換し，末尾に CRC を自動計算して格納する.
- **引数**：
  - `uint8_t* buf`：変換先の 8 バイトバッファ
- **戻り値**：なし
- **備考**：CRC は flag〜data の 6 バイトに対して計算される.

5 名前空間 `Protocol::Factory`

5.1 void createButtonPressMessage(uint8_t* buffer, uint32_t time)

- **概要**：ボタン押下を示すメッセージを作成する.
- **引数**：
  - `uint8_t* buffer`：生成したメッセージを格納する 8 バイト配列
  - `uint32_t time`：ボタン押下時刻のタイムスタンプ
- **戻り値**：なし

5.2 void createLEDStateRequest(uint8_t* buffer)

- **概要**：LED 状態のリクエストメッセージを作成する.
- **引数**：
  - `uint8_t* buffer`：生成したメッセージを格納する 8 バイト配列
- **戻り値**：なし

5.3 void createLEDControlResponse(uint8_t* buffer, uint32_t time, bool turnOn)

- **概要**：LED の ON/OFF 制御レスポンスメッセージを作成する.
- **引数**：
  - `uint8_t* buffer`：生成したメッセージを格納する 8 バイト配列
  - `uint32_t time`：LED の変更時間指定
  - `bool turnOn`：LED を ON にする場合は true，OFF にする場合は false
- **戻り値**：なし

5.4 void createTimeSyncRequest(uint8_t* buffer, uint32_t t1)

- **概要**：時刻同期リクエストメッセージを作成する.
- **引数**：
  - `uint8_t* buffer`：生成したメッセージを格納する 8 バイト配列
  - `uint32_t t1`：リクエスト送信時刻のタイムスタンプ
- **戻り値**：なし

5.5 void createTimeSyncResponse(uint8_t* buffer, uint32_t t3, uint8_t diff)

- **概要**：時刻同期レスポンスメッセージを作成する.
- **引数**：
  - `uint8_t* buffer`：生成したメッセージを格納する 8 バイト配列
  - `uint32_t t3`：レスポンス送信時刻のタイムスタンプ
  - `uint8_t diff`：時刻差（t2 - t1）
- **戻り値**：なし

6 名前空間 Protocol::Parser

6.1 enum class ParseResult

- **概要**：メッセージ解析結果の列挙型
- **値**：
  - `OK`：解析成功
  - `NotEnoughData`：データ長不足（8 バイト未満）
  - `UnknownFlag`：未知のフラグ検出
  - `CRCMismatch`：CRC 不一致

6.2 ParseResult tryParseMessageFromBuffer(const uint8_t* buffer, size_t len, ProtocolMessage& msg)

- **概要**：バッファからプロトコルメッセージを解析し，構造体に展開する．
- **引数**：
  - `const uint8_t* buffer`：解析対象のバイト列
  - `size_t len`：バッファ長
  - `ProtocolMessage& msg`：データ格納先
- **戻り値**：`ParseResult` 列挙型の解析結果コード
- **備考**：バッファ長が 8 未満の場合やフラグが未知の場合，CRC が一致しない場合にそれぞれエラーを返す．

6.3 MessageType getMessageType(const ProtocolMessage& msg)

- **概要**：メッセージの flag 値に基づきメッセージタイプを判別する．
- **引数**：
  - `const ProtocolMessage& msg`：判別対象のメッセージ
- **戻り値**：メッセージの種類を示す `MessageType` 列挙型値